

Generalization

By

Prof. Arpit Agrawal and Prof. Rajdip Nayek

Assistant Professor,
Department of Applied Mechanics,
IIT Delhi

IIT Delhi × Bechtel AI/ML Programme | 25 June 2026

<https://rajdip-iitd.github.io/bechtel2/>

Email: arpit@am.iitd.ac.in &
rajdipn@am.iitd.ac.in

Introduction

- In the last two lectures, we talked on how to optimize the parameters of our neural network model using the **training data**
- However, we don't just want our model to only fit the training data
- We want it to ***generalize*** to data that our model has not seen before
- A convenient way of measuring a model's generalization performance is to measure its performance on a held-out **test set**, which consist of examples it hasn't seen before.
- If a model works well on the training set but fails to do well on test data, we say it is **overfitting**

This lecture will cover a few simple strategies to prevent overfitting

Learning goals

- Difference between **training** set, **validation** set, and **test** set
- Be able to reason qualitatively about how training and test error depend on
 - the size of the model
 - the number of training examples
 - the number of training iterations (or epochs)
- Understand the motivation behind using different strategies:
 - Data augmentation
 - Reducing network parameters
 - Weight decay
 - Early stopping
 - Ensembles
 - Dropout

How to measure generalization performance

- We partition our dataset into three subsets
- A **training set** ← a set of training examples the network is trained on
- A **validation set** ← used to tune hyperparameters such as number of hidden units, or the learning rate, or the hyperparameters of optimizers, etc.
- A **test set** ← used to [measure the generalization performance](#) of final trained model



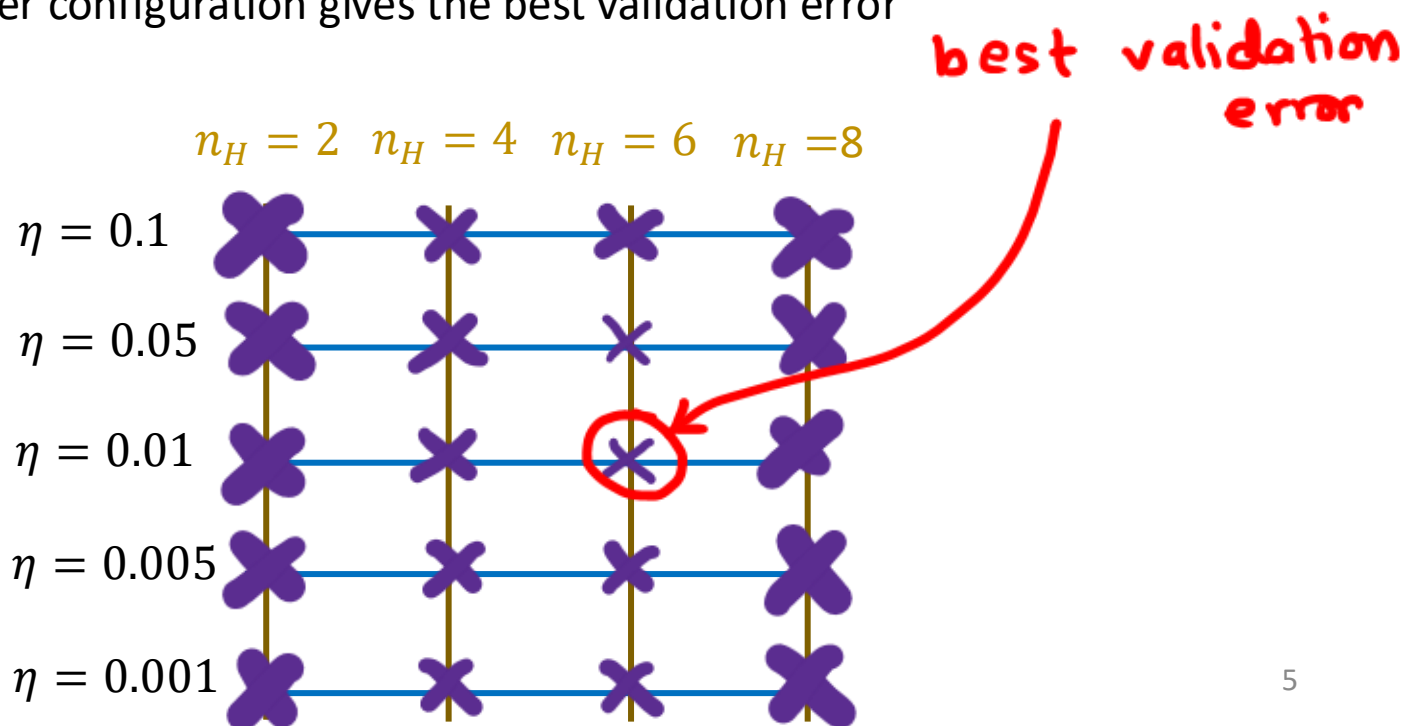
- The losses on these respective subsets are called **training loss**, **validation loss**, and **test loss**

Basic strategies of tuning hyperparameters

How to tune hyperparameters using validation set?

Grid search

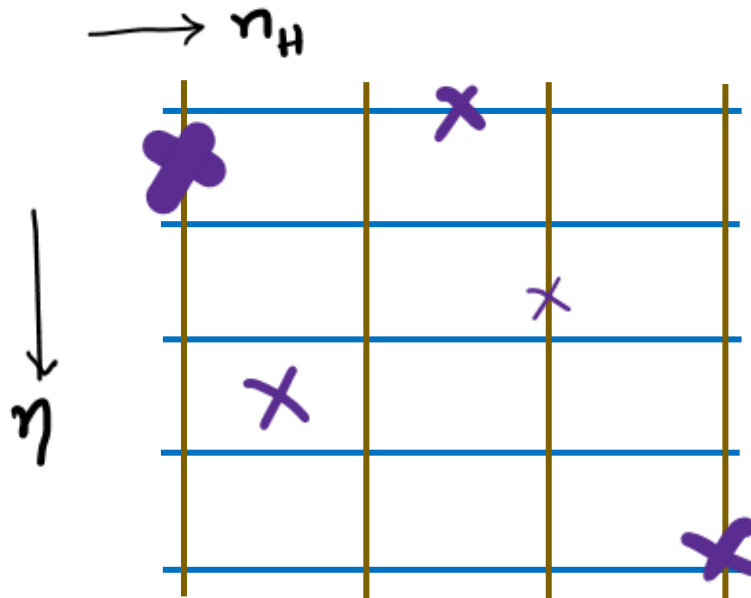
- For each hyperparameter, choose a set of candidate values
 - E.g., choose learning rate, $\eta = 0.1, 0.05, 0.01, 0.005, 0.001$
 - E.g. choose number of hidden units in a layer H , $n_H = 2, 4, 6, 8$
- Separately train models using all possible configurations of these values
- Choose whichever configuration gives the best validation error



Basic strategies of tuning hyperparameters

Random search

- Train a bunch of neural networks using random configurations of hyperparameters
 - E.g., sample a set of learning rates uniformly from $U(0.001, 0.1)$
 - E.g. sample random integers for number of hidden units in a layer between 1-10
 - E.g. configurations $\rightarrow (0.0013, 2), (0.041, 5), (0.02, 8)$, etc.
- Separately train models using the sampled configurations and choose whichever configuration gives the best validation error



When can we generalize poorly?

- Training data distribution $\neq$ Test data distribution
- Model has **too much capacity** (or too less capacity)

When can we generalize poorly?

- It is assumed that the training set distribution is same as the test set distribution
- If the training set distribution differs from the test set distribution, the model will generalize poorly on the test set

Training set

Test set

- Ex:



- Ex:

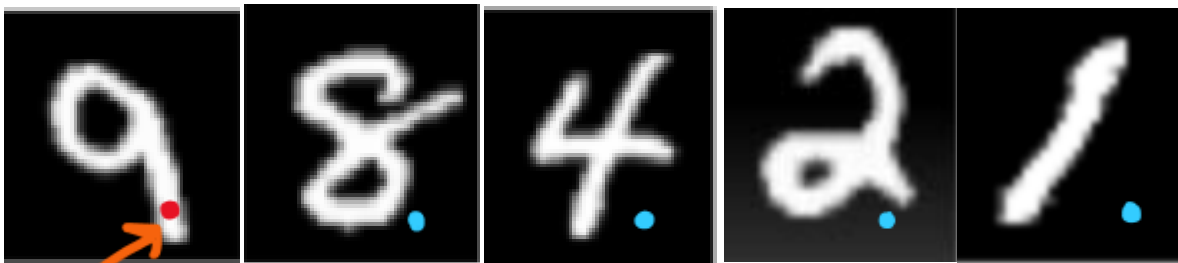


When can we generalize poorly?

- Most often in neural networks, if a network performs well on the training set but generalizes badly, it is probably due to **overfitting**
- A network might overfit if the training set contains **accidental regularities** — features that are not true representatives of the output labels
- For example, in MNIST digit classification, it might so happen that in the training set, all images of 9's have pixel number 622 on, while all other examples have it off. The network might decide to exploit this accidental regularity, thereby correctly classifying all the training examples of 9's, without learning the true regularities. If this property doesn't hold on the test set, the network will generalize badly

Training dataset

Test data



Pixel # 622

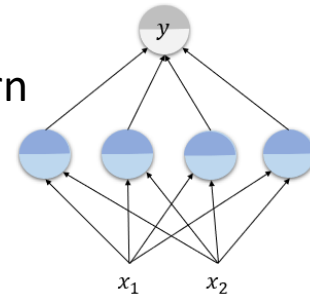
When can we generalize poorly?

- **Class imbalance** is another problem that can occur in classification
- Very similar to the problem: training data distribution $\neq$ test data distribution
- It refers to a problem where the number of examples in the training dataset for each class label is not balanced
- Example, we may have 80 examples of one flower species and 20 examples of a second flower species in our training set



When can we generalize poorly?

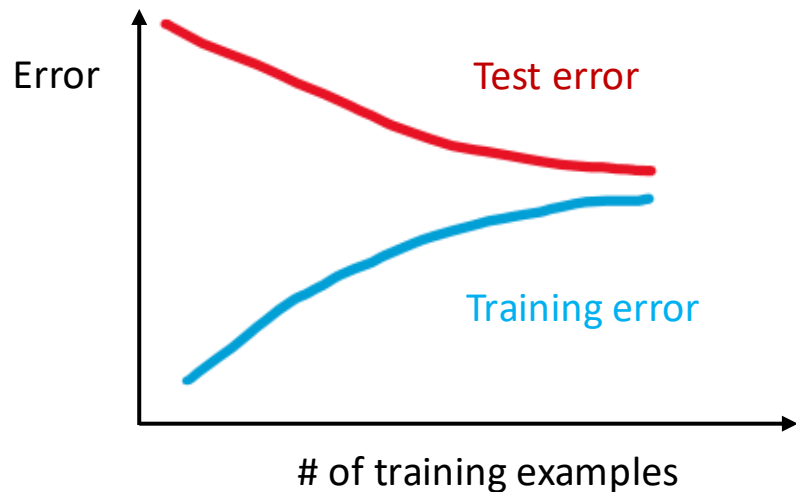
- Another extreme case of overfitting can happen when the neural network **memorizes** the training data
- Recall the neural network we constructed in Lecture that was able to learn arbitrary Boolean functions
- It had a separate hidden unit for every possible input configuration. This network architecture is able to **memorize** a training set, i.e. learn the correct answer for every training example
- The problem is that this network has too large a **capacity**, i.e. ability to remember information about its training data
- Capacity isn't a formal term, but corresponds roughly to the number of trainable parameters (i.e. weights). Information is stored in the network's parameters, so networks with more parameters can store more information



- To this end, we saw
 - How to measure generalization error → using test data
 - When do we get large generalization errors
 - Test distribution $\neq$ train distribution
 - Accidental regularity
 - Class imbalance
 - Overfitting due to very large capacity
- Now let's see, how does generalization error vary with
 - (a) the number of training examples,
 - (b) the number of model parameters

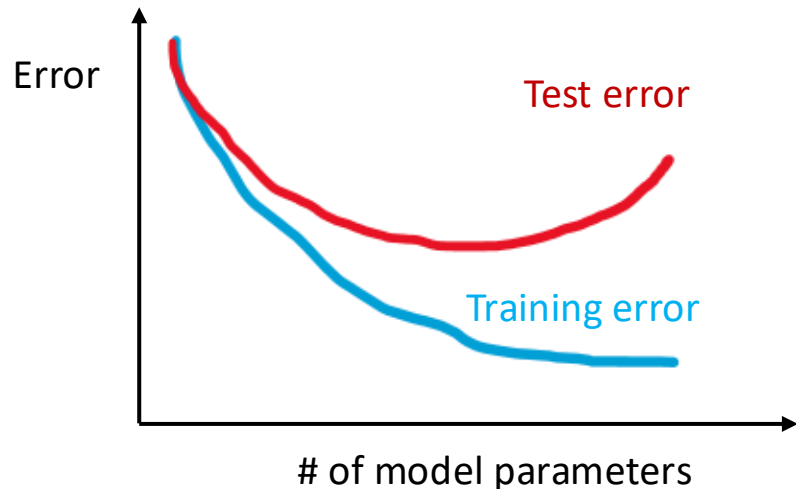
Generalization error vs # of training examples

- What happens to the training and test error as you increase training examples?
- For any particular test example, if you have a larger training set, there are more chances that the test example will be closely related to a training example
- With a larger training set, there would be fewer accidental regularities and more balanced data distribution, hence test error will decrease
- What happens to training error?
- Small training sets are easier to memorize than large ones, so training error tends to increase as we add more examples
- Eventually both the training and test error meet close to each other



Generalization error vs # of model parameters

- What happens to the training and test error as you increase model parameters?
- As we add more parameters in the model (e.g. add more units), it becomes easier to fit both the accidental and the true regularities of the training data. Therefore, training error reduces as we add more parameters
- The test error first decreases and then increases
 - If the network has too little capacity, it generalizes badly because it fails to pick up the regularities (true or accidental) in the data – **underfitting**
 - If it has too much capacity, it will memorize the training set -- **overfitting**
- We would like to design network architectures that will have enough capacity to learn the true regularities in the training data, but not enough capacity to simply memorize the training set or exploit accidental regularities



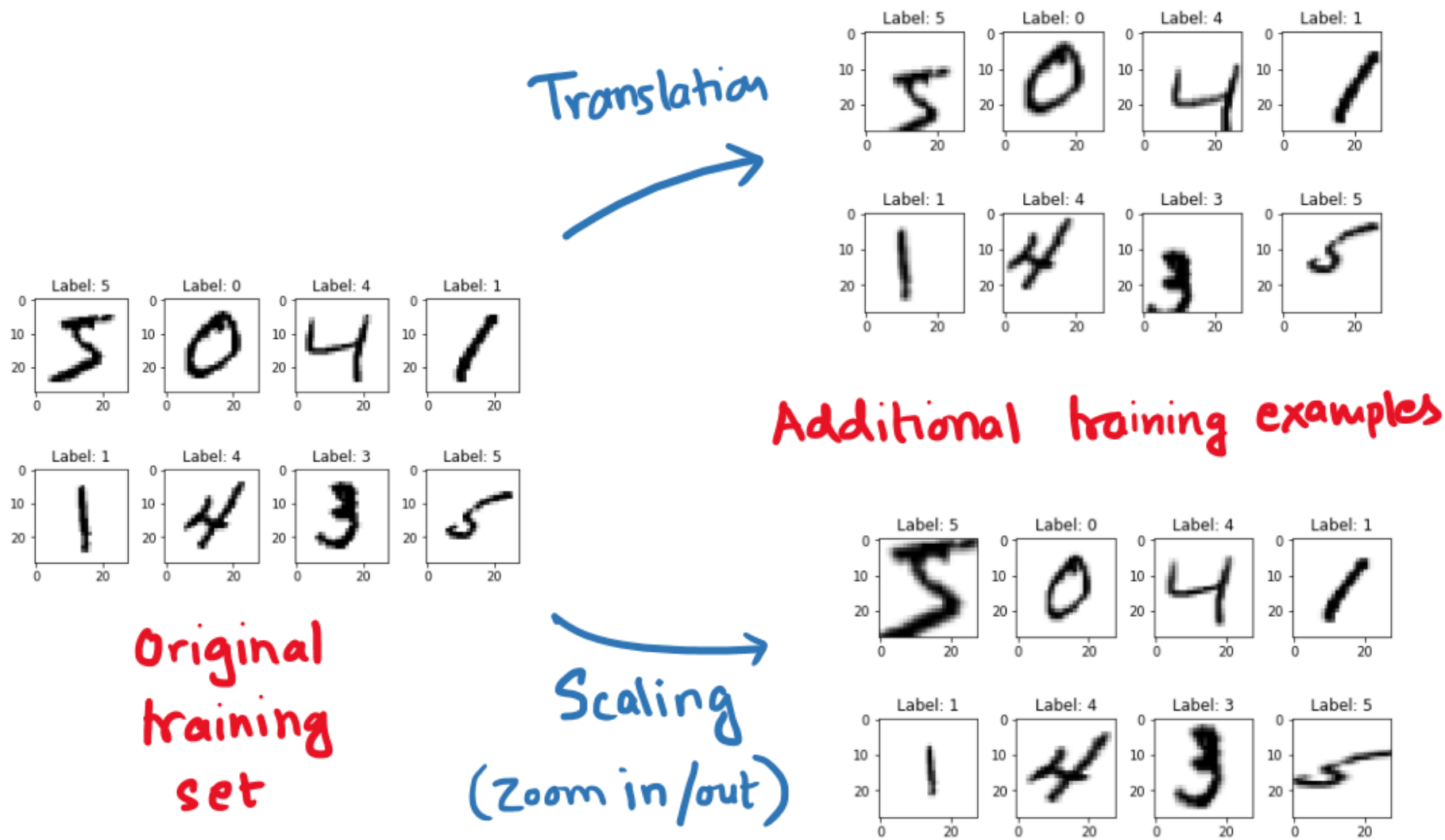
Tricks to improve generalization

- How can we train a model that has enough capacity to fit the data, but does not overfit?
- Add more training data or reduce the parameters?
- Tricks we will see in the rest of the lecture
 - data augmentation
 - reducing the number of parameters
 - weight decay
 - early stopping
 - ensembles (combine predictions of different models)
 - dropout
- It is often a good practice **not to use** any of the aforementioned regularization techniques until we see/think there is overfitting during training

Data augmentation

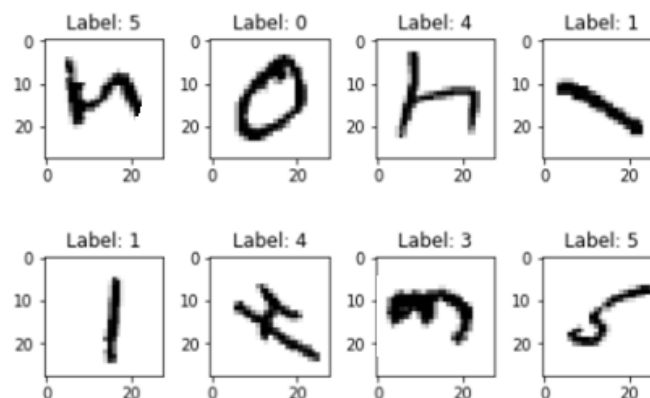
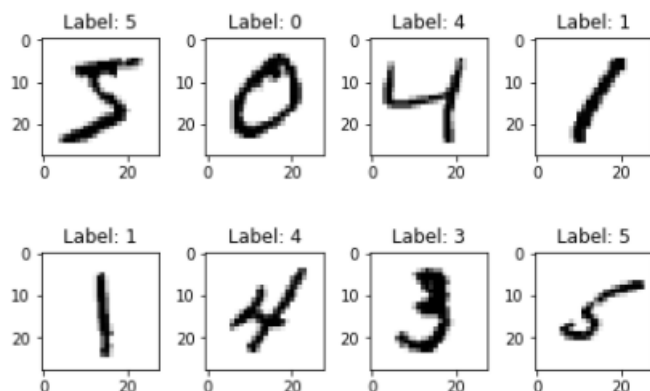
Data augmentation

- We can **augment** the training data by artificially creating new training examples from the existing training examples



Data augmentation (images)

Large rotations



For the MNIST dataset, large rotations by $90^\circ/180^\circ$ would be disastrous!

Data augmentation

- We can **augment** the training data by artificially creating new training examples from the existing training examples
- The **choice of transformations depends on the task** (e.g. horizontal flip for object recognition, but not handwritten digit recognition)

Examples (for images)

- Translation
- Flipping
- Rotation
- Smooth warping
- Add noise (e.g. flip random pixels)

Examples (for time-series)

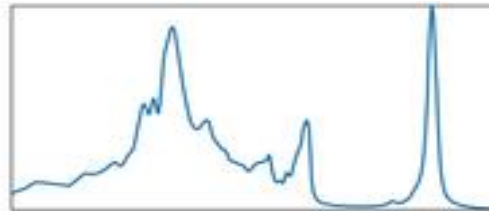
- Scaling
- Flipping
- Window slicing
- Magnitude warping
- Jittering

- **Only modify the training examples**, not the test examples

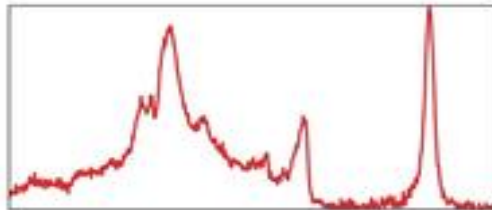
<https://iq.opengenus.org/data-augmentation/>

Data augmentation (time-series)

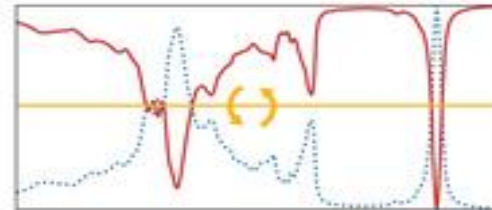
*An empirical survey of data augmentation for time series classification with neural networks (2021)



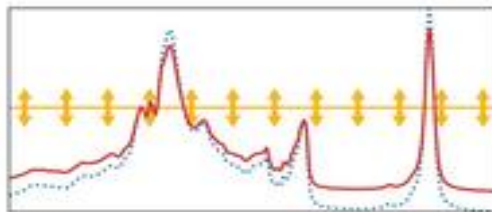
(a) Original



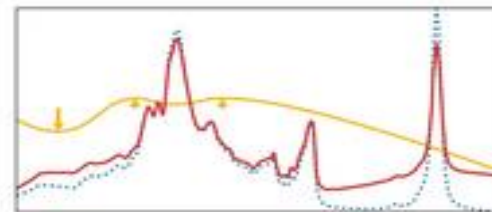
(b) Jittering



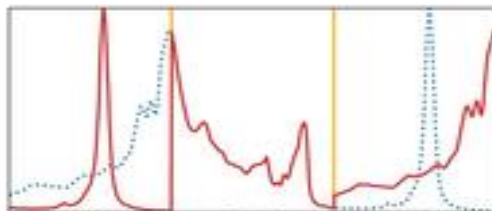
(c) Flipping



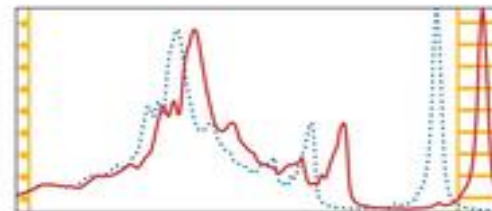
(d) Scaling



(e) Magnitude Warping |



(f) Permutation

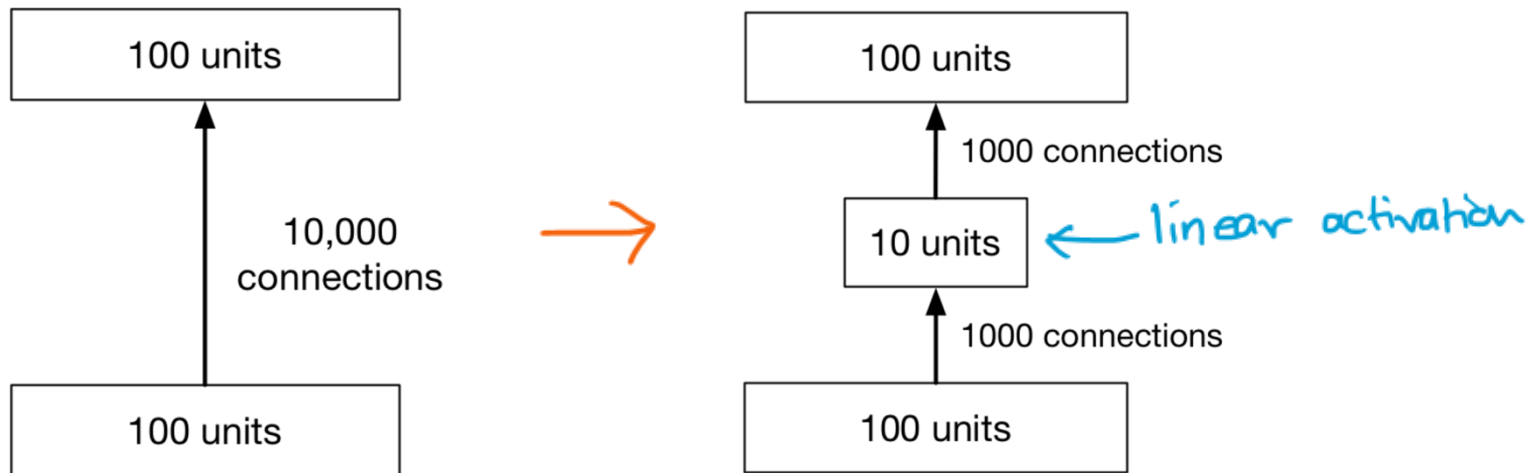


(g) Window Slicing

Reducing the number of
parameters / Parameter sharing

Reducing the number of model parameters

- Can we reduce the size of the network like the number of layers or the number of units per layer?
- These hyperparameters can be tuned on a validation set (in order to find a model, which has enough capacity to learn the true regularities, but not enough to overfit)
- One strategy to reduce the number of parameters is by adding a **linear bottleneck layer**. This is a layer with fewer units than the layers below or above it

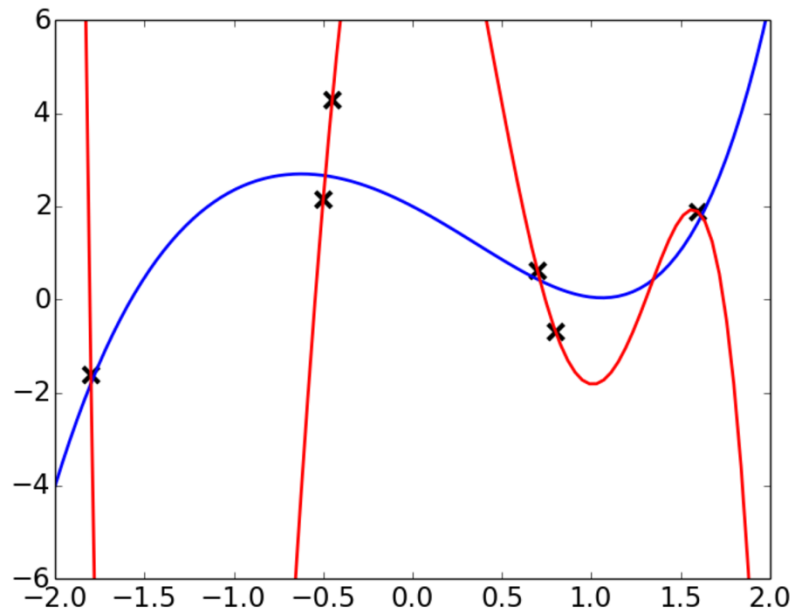


- **Drawback:** Reducing the capacity might make the network too simple to learn the true regularities in the data

Weight decay (regularization)

Weight decay

- One way of penalizing model capacity (or complexity) while still keeping a lot of parameters is to encourage the **parameters (weights) to be small in magnitude**
- Why we want the weights to be small?



$$y = 0.1x^5 + 0.2x^4 + 0.75x^3 - x^2 - 2x + 2$$

$$y = -7.2x^5 + 10.4x^4 + 24.5x^3 - 37.9x^2 - 3.6x + 12$$

The red polynomial overfits! Note that it has really large coefficients

Weight decay

- **How can we make the weights small? Idea:** Include them in the loss function
- We now add a **regularizer** to the loss function, which **penalizes the weights**
 - Most often, penalization is only applied to weights, and not the biases
 - However, PyTorch applies weight decay to both weights and biases by default

$$\mathcal{L}(\boldsymbol{\theta}) = \underbrace{\frac{1}{N} \sum_{i=1}^N \ell(\mathbf{y}(\mathbf{x}^{(i)}; \boldsymbol{\theta}), \mathbf{t}^{(i)})}_{\text{training loss}} + \underbrace{\mathcal{R}(\boldsymbol{\theta})}_{\text{regularizer}}$$

- $\boldsymbol{\theta}$ is a vector containing all weights and biases of a neural network

Weight decay (L_2 - regularization)

- L_2 -regularization

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{y}(\mathbf{x}^{(i)}; \boldsymbol{\theta}), \mathbf{t}^{(i)}) + \mathcal{R}(\boldsymbol{\theta})$$

$$\mathcal{R}_{L_2}(\boldsymbol{\theta}) = \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2 = \frac{\lambda}{2} \sum_q \theta_q^2$$

- It penalizes the sum of squares of all the weights in all layers of the network
- The hyperparameter $\lambda \geq 0$ is called the **regularization constant**
- L_2 regularization places an huge penalty on large weights (because of the square term). This biases the optimizer towards models that distribute weight evenly across all features

Weight decay (L_2 - regularization)

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \ell^{(i)}(\boldsymbol{\theta}) + \mathcal{R}_{L_2}(\boldsymbol{\theta}) \quad \mathcal{R}_{L_2}(\boldsymbol{\theta}) = \frac{\lambda}{2} \sum_q \theta_q^2$$

- It is pretty straightforward to incorporate L_2 -regularizers into Gradient Descent (GD) computations. In particular, by linearity of derivative

$$\frac{\partial \mathcal{L}}{\partial \theta_j} = \frac{1}{N} \sum_{i=1}^N \frac{\partial \ell^{(i)}}{\partial \theta_j} + \frac{\partial \mathcal{R}_{L_2}}{\partial \theta_j}$$

- E.g. for SGD, we would have the following update procedure

$$\theta_j \leftarrow \theta_j - \eta \left(\frac{\partial \ell^{(i)}}{\partial \theta_j} + \frac{\partial \mathcal{R}_{L_2}}{\partial \theta_j} \right)$$

$$\theta_j \leftarrow \theta_j - \eta \left(\frac{\partial \ell^{(i)}}{\partial \theta_j} + \lambda \theta_j \right)$$

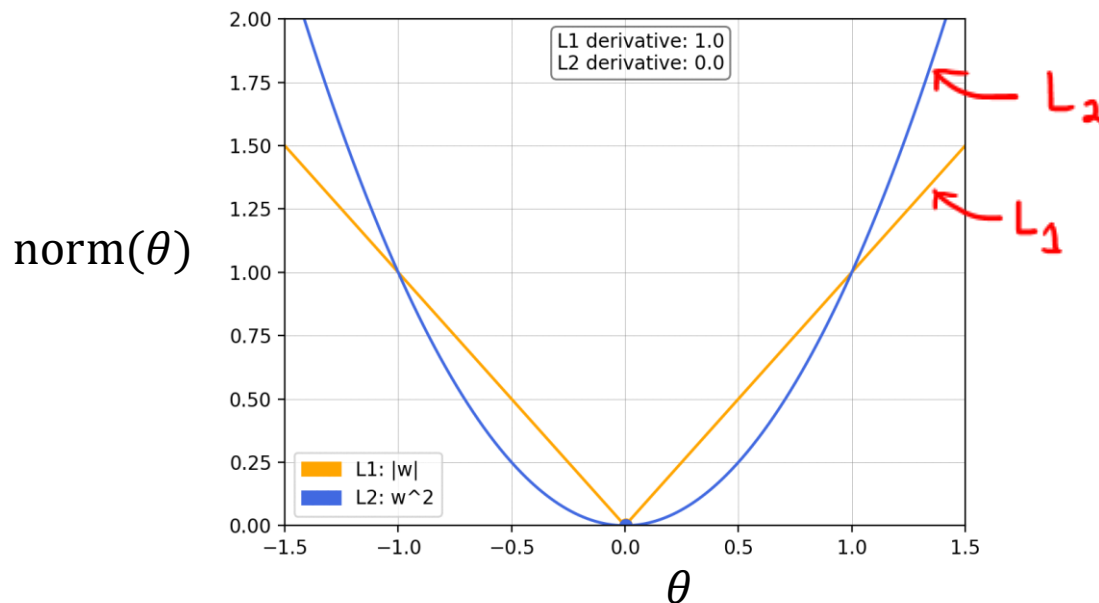
$$\theta_j \leftarrow (1 - \eta\lambda)\theta_j - \eta \frac{\partial \ell^{(i)}}{\partial \theta_j}$$

In each iteration, we shrink the weights by a factor of $(1 - \eta\lambda)$. For this reason, L_2 regularization is also known as **weight decay**

```
optimizer = torch.optim.SGD(model.parameters(), lr=1e-3, weight_decay=1e-2)
```

Other types of weight decay

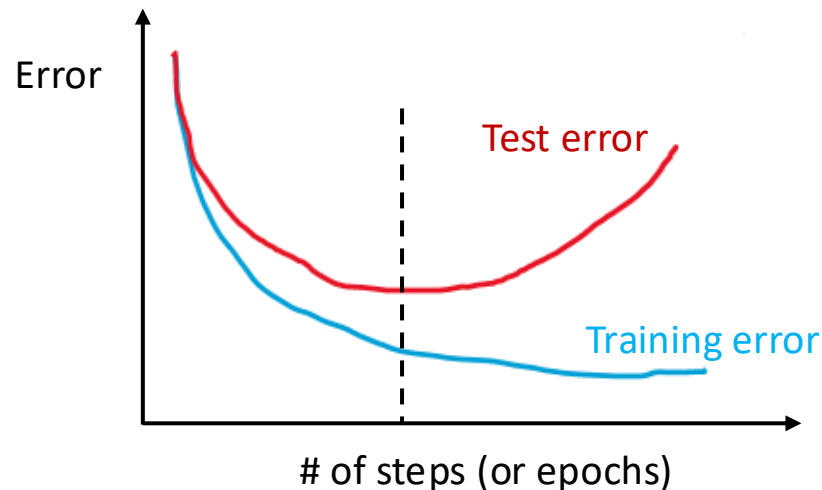
- There are other kinds of regularizers which encourage weights to be small, e.g. sum of the absolute values (L_1 - regularization)
- Regularizers differ by how strongly they prioritize making weights exactly zero, versus not being very large
- These alternative penalties are commonly used in other areas of machine learning, but less commonly for neural nets



Early stopping

Early stopping

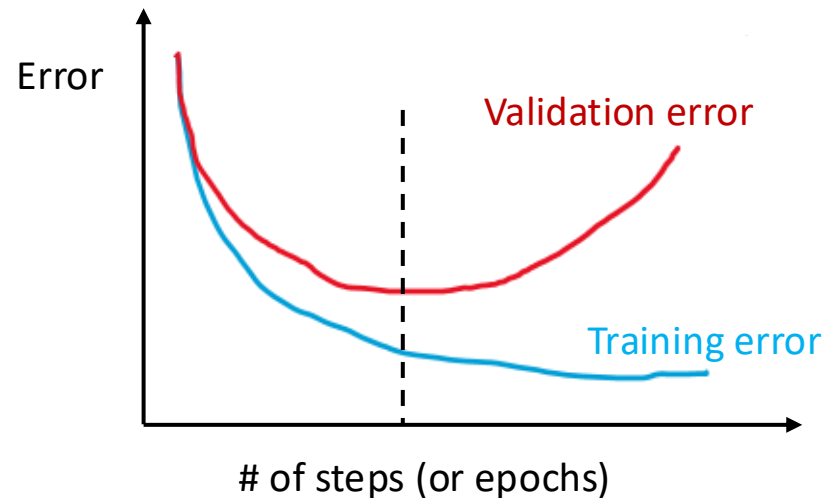
- How long should we train our neural networks to prevent overfitting?
- While training (i.e. optimizing), the training error continues to decrease with more number of steps (or epochs)
- The test error generally improves at first, but it may eventually start to increase as the network starts to overfit



- This suggests an obvious strategy – **early stopping**: stop the training at the point where the generalization error starts to increase

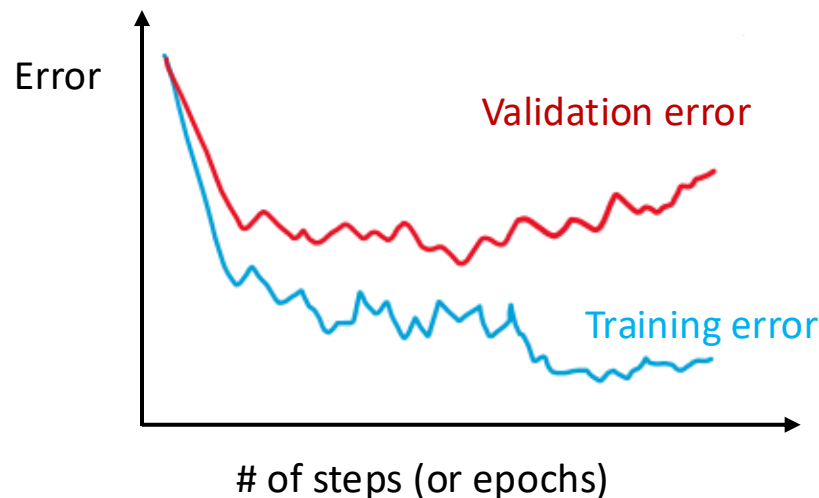
Early stopping

- But we can't do early stopping using the test set, because that would be cheating
- Therefore, we would determine when to stop by creating a validation set and monitoring the validation error during training



Early stopping

- Unfortunately, implementing early stopping is a bit harder than it looks, **why?**
- The reason is that the training and validation errors fluctuate during training (because of stochasticity in the gradients)
- It can be hard to tell whether an increase is simply due to these fluctuations

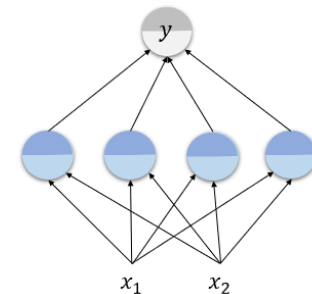


- One common heuristic is to print the validation errors once every 2 epochs. If the validation error fails to improve after every two epochs (or perhaps after several consecutive epochs), then stop training

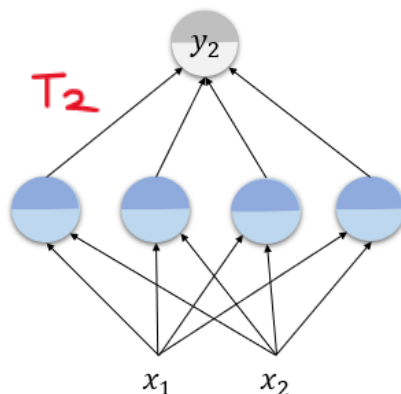
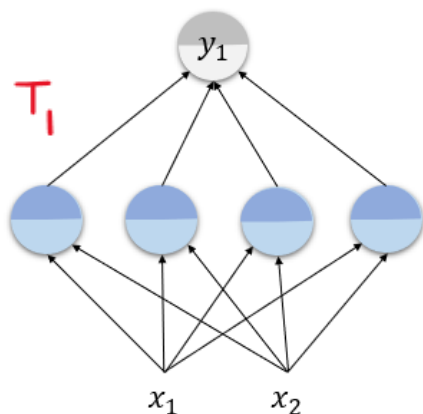
Ensemble methods

Ensemble methods (Bagging)

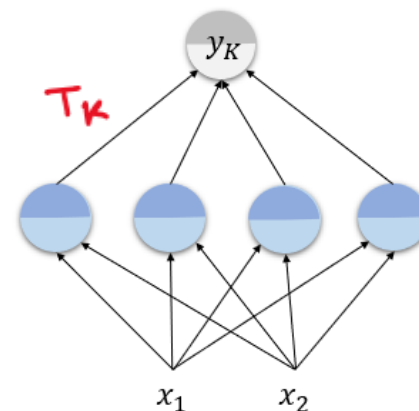
- Combining predictions of different models can reduce generalization error
- Bagging:** Train on random subsets of the full training dataset
- Create a total of K instances of the same model; K is a hyperparameter
- Construct multiple training subsets by sampling from the full training dataset with replacement. Subsets $\rightarrow T_1, T_2, \dots, T_K$
- Train the k th instance with training data T_k
- Use the average of the predicted output from all K instances



$$y_{\text{bag}} = \frac{1}{K} \sum_{k=1}^K y_k$$



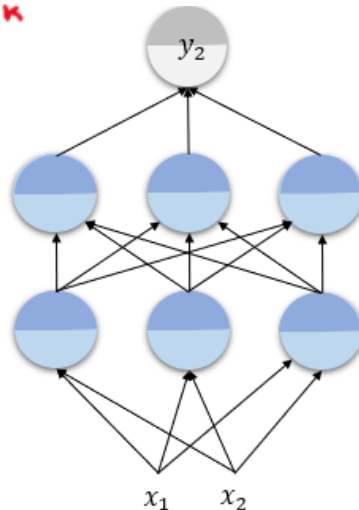
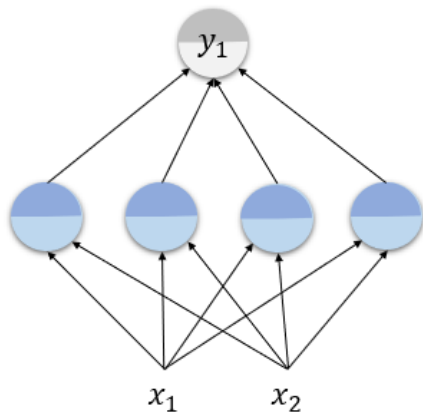
...



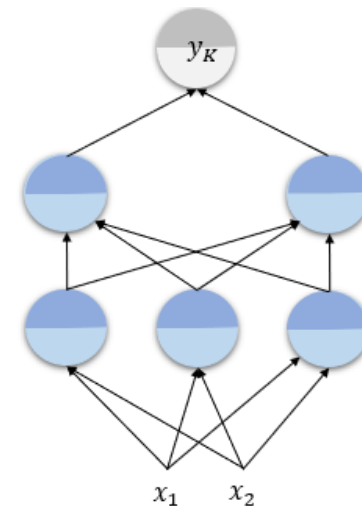
Ensemble methods

- Combining predictions of different models can reduce generalization error
- Train using different architectures of neural network on the training data
- Create a total of K different neural network models and train all K models using full training data
- Use the average of the predicted output from all K different models

$$y = \frac{1}{K} \sum_{k=1}^K y_k$$

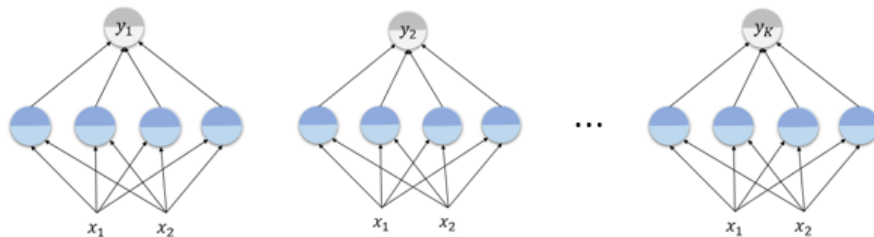


...

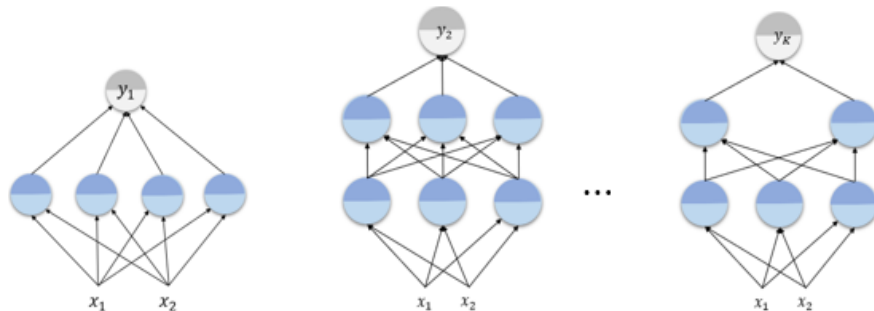


Dropout

Dropout



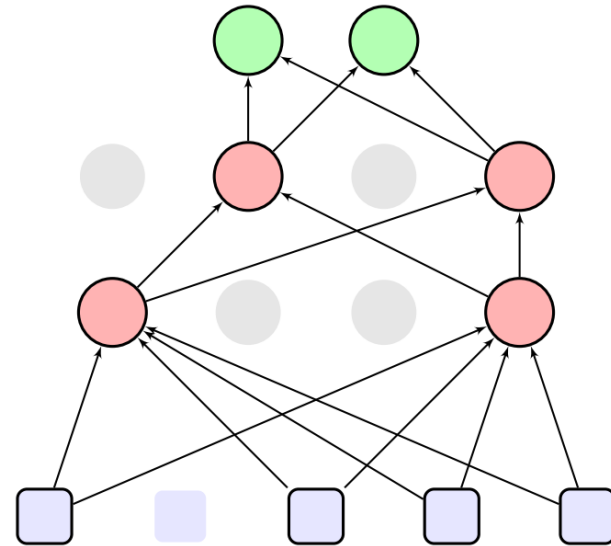
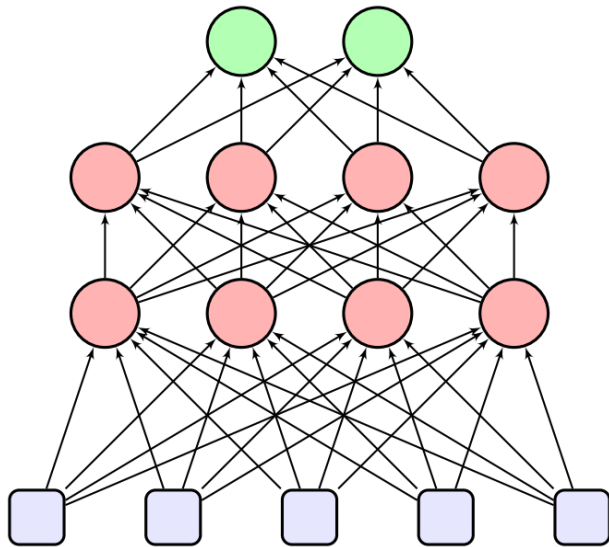
Option 1: Train multiple instances of the same network using different training samples (expensive)



Option 2: Train several neural networks having different architectures (obviously expensive)

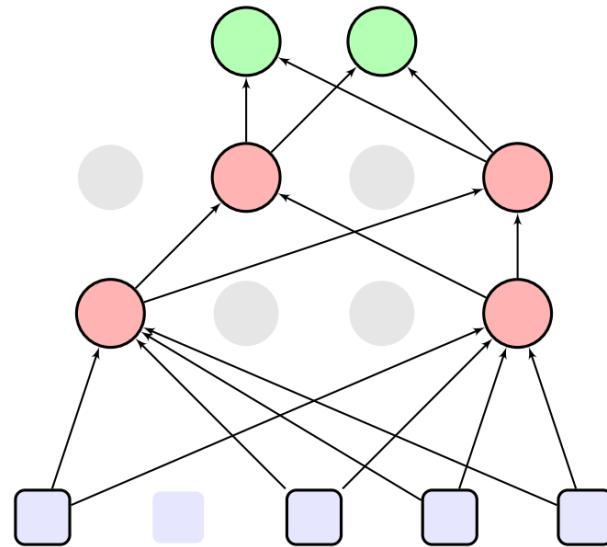
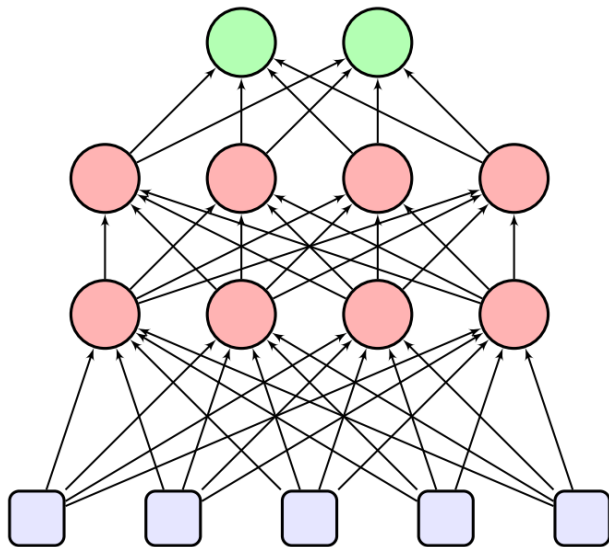
- Typically, model averaging (ensembles) is always better for generalization
- However, training several large neural networks for making an ensemble is prohibitively expensive
- **Dropout** is a technique which addresses both these issues
- Effectively it allows training several neural networks without any significant computational overhead

Dropout



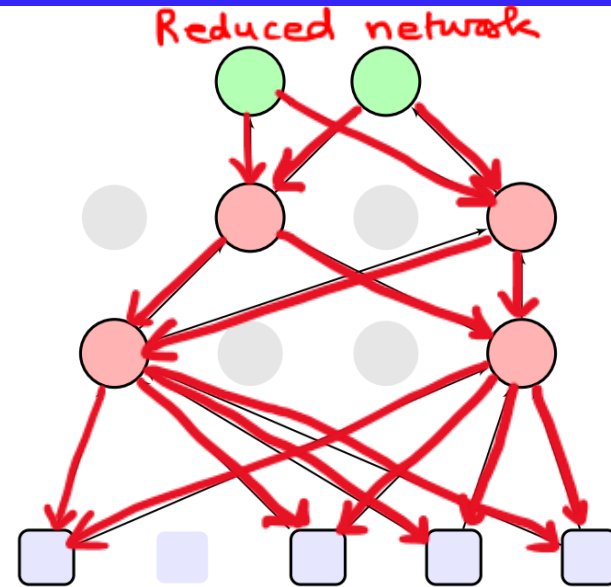
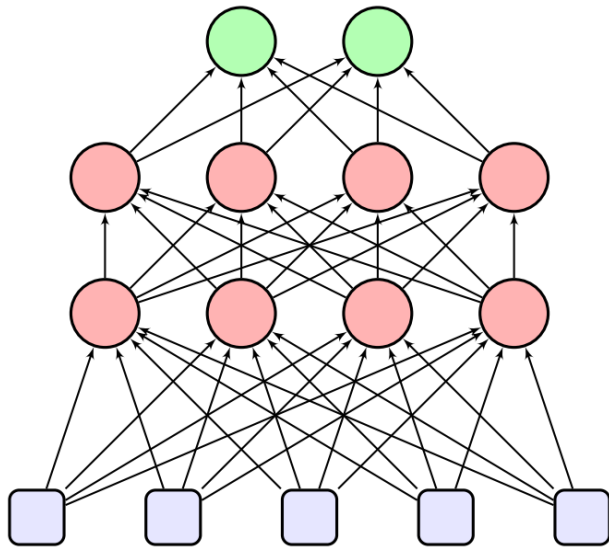
- Dropout refers to **dropping out of units**. Temporarily remove a unit/node and all its incoming/outgoing connections resulting in a reduced network
- Dropout may be implemented on any or all hidden layers in the network as well as the visible or input layer. It is not used on the output layer
- Each node is dropped with a fixed probability ρ (which is a hyperparameter). Typically $\rho = 0.5$ for hidden nodes and $\rho = 0.2$ for input nodes

Dropout



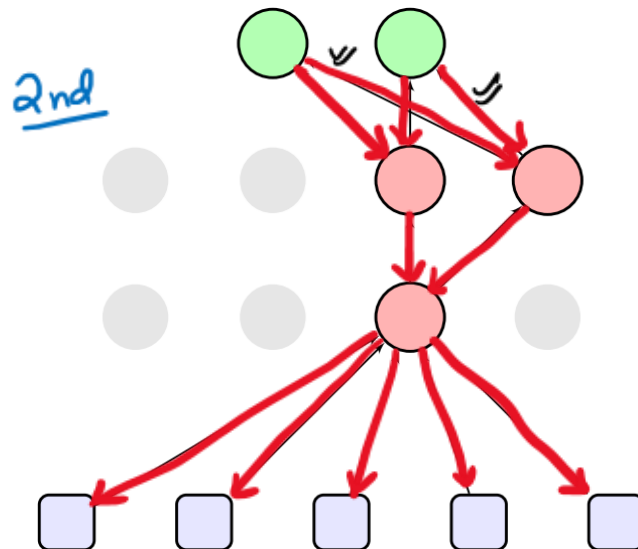
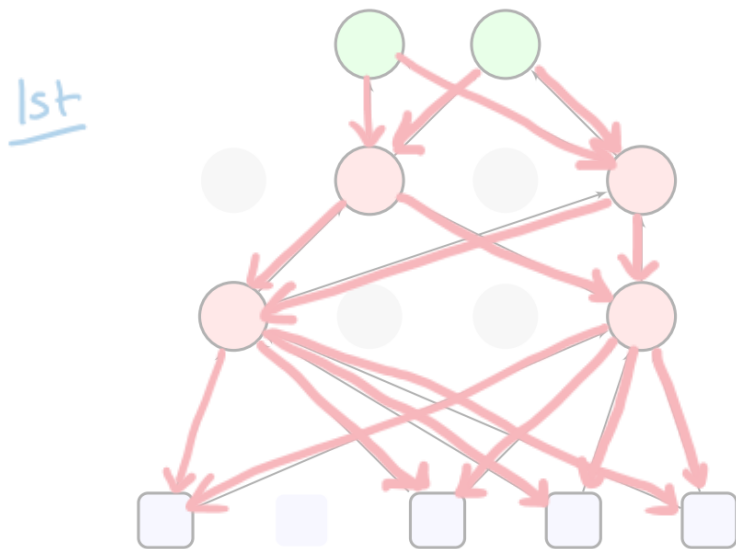
- For the above example, we dropped 5 nodes to get a reduced network
- Given a total of M (hidden+input) nodes of the network, there are a total of 2^M possible reduced networks
- Training all possible networks is computationally prohibitive
- **Trick:** Share the weights across all the networks

Dropout



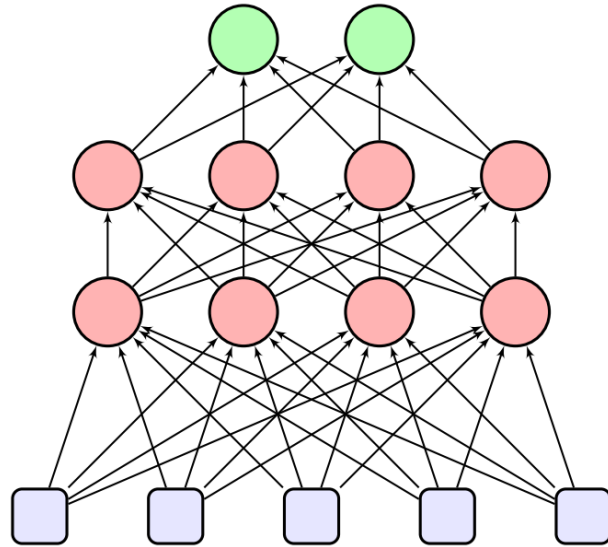
- We initialize all the parameters (weights) of the network and start training
- For the first training instance (or mini-batch), we apply dropout resulting in a reduced network
- We compute the loss and do backpropagation
- Which parameters do we update?
- Only those that are active

Dropout



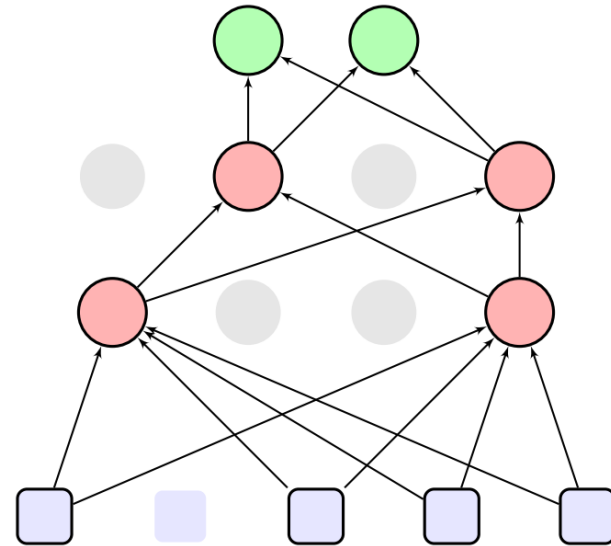
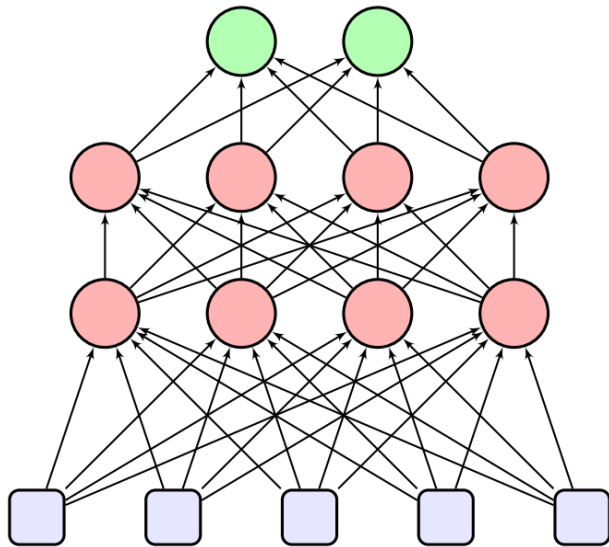
- For the second training instance (or mini-batch), we apply dropout again resulting in a different reduced network
- We again compute the loss and do backpropagation for the active weights
- If the weight was active for both the training instances then it would have received two updates by now whereas other would receive one update
- Note that all of the 2^M possible reduced networks rarely gets trained ever but the parameter sharing ensures all the weights get trained for the fully-connected model

Dropout



- After training is completed, all the weights are multiplied by a factor $(1 - \rho)$
- **Reason:** The weights of the network will be larger than normal because of dropout. Therefore, before finalizing the network, the weights are first scaled by the chosen dropout rate
- What happens during testing?
- No dropout during testing
- Just use the fully connected network for prediction – acts like model averaging

Dropout



- What happens during testing?
- Impossible to aggregate the outputs of 2^M reduced networks
- Only those that are active